

PROYECTO FIN DE CICLO

Ciclo Formativo de Grado Superior

Desarrollo de Aplicaciones Multiplataforma (2º Curso)

PARKABLE

Smart Parking. Shared Spaces.

Aplicación Android nativa para el alquiler colaborativo de plazas de garaje y avisos de aparcamiento en la calle

Alumno:

Antonio Raño

Centro:

IES El Majuelo

Curso académico 2025-2026

Índice

1. Introducción.....	4
1.1. Objetivos del proyecto.....	4
1.2. Justificación del enfoque técnico.....	4
1.3. Alcance del documento	5
2. Requisitos para el Desarrollo	6
2.1. Requisitos principales	6
Hardware mínimo recomendado.....	6
Sistema operativo	6
Software necesario	6
Servicios en la nube	6
2.2. Requisitos opcionales.....	6
2.3. Dependencias.....	6
Servicios externos	7
Orden de inicialización	7
3. Puesta en marcha de la aplicación y uso	8
3.1. Estructura del código y archivos	8
Capas y responsabilidades	8
3.2. Conceptos clave	9
Kotlin + Jetpack Compose	9
Firebase como backend serverless	9
Google Maps Compose	9
DataStore Preferences	9
Sistema de locales personalizado	9
Imágenes en Base64 dentro de Firestore	9
3.3. Base de Datos (Esquema).....	10
Colección users	10
Colección listings.....	10
Colección bookings	11
Colección parking_alerts.....	11
Colección points_transactions	11
Diagrama relacional simplificado	12
Modo de seguridad	12
3.4. Procesos principales.....	12
Flujo 1: registro y autenticación	12
Flujo 2: publicación de un anuncio	12
Flujo 3: reserva con pago simulado	13

Flujo 4: aviso SocialDrive con confirmación bilateral y entrega de puntos	13
Flujo 5: cambio de idioma en caliente	13
Flujo 6: canje de recompensas.....	13
4. Posibles ampliaciones	14
4.1. Ampliaciones técnicas.....	14
4.2. Ampliaciones funcionales	14
4.3. Ampliaciones de gamificación.....	14
5. Valoración personal	15
5.1. Mayores dificultades encontradas.....	15
Cambio de idioma en caliente	15
Mantener todo en el plan gratuito	15
Doble confirmación bilateral	15
Geocoding gratuito	15
5.2. Aprendizajes clave	16
5.3. Conclusión.....	16
6. Apéndices.....	17
6.1. Material adicional incluido.....	17
6.2. Pasos para reproducir el proyecto.....	17
6.3. Tecnologías y sus versiones (resumen).....	17
6.4. Referencias y enlaces de interés.....	18
6.5. Contacto.....	18

1. Introducción

Parkable es una aplicación Android nativa que aborda dos problemáticas reales del entorno urbano: la dificultad para encontrar plazas de aparcamiento y la infrautilización de garajes privados. La aplicación combina un marketplace de plazas (al estilo de plataformas como Wallapop o MilAnuncios, pero especializado en garajes) con un foro de avisos en tiempo real (inspirado en SocialDrive) que permite a los conductores comunicar plazas que se quedan libres en la vía pública.

El proyecto nace de un análisis sectorial previo del ecosistema TIC andaluz, dentro del que se identificó una oportunidad clara en el ámbito de la movilidad urbana inteligente. Aplicando una metodología de priorización (Urgencia / Alcance / Impacto), se seleccionó la oportunidad "Sistema de reserva inteligente con geolocalización y pagos integrados" como la más prometedora. Sobre esta base se ha diseñado e implementado la aplicación que se documenta en este informe.

1.1. Objetivos del proyecto

El objetivo general es desarrollar una aplicación Android funcional, lista para presentar como producto mínimo viable (MVP), que demuestre el dominio de los contenidos del ciclo: programación con Kotlin, interfaces con Jetpack Compose, persistencia y sincronización en la nube, geolocalización, multilingüismo y arquitectura limpia.

Como objetivos específicos se han establecido:

- Implementar un marketplace funcional de plazas de garaje con publicación, búsqueda en mapa o lista, detalle, reserva y pago simulado.
- Implementar un foro de avisos en tiempo real con confirmación bilateral entre conductor que sale y conductor que llega.
- Diseñar un sistema de gamificación basado en puntos, con catálogo de recompensas canjeables, que incentive el uso colaborativo de la plataforma.
- Soportar dos idiomas (español e inglés) con cambio en caliente.
- Mantener todo el backend dentro del plan gratuito de Firebase (Spark) para que el proyecto sea reproducible sin coste.

1.2. Justificación del enfoque técnico

Se ha optado por una aplicación nativa Android (Kotlin + Jetpack Compose) frente a alternativas multiplataforma por dos razones: primera, el ciclo formativo es de Desarrollo de Aplicaciones Multiplataforma con especialización Android, y se considera más adecuado profundizar en la plataforma principal antes de abstraerla; segunda, las funcionalidades clave (geolocalización en tiempo real, cámara para fotos del anuncio, notificaciones locales) tienen mejor soporte y rendimiento en código nativo.

El backend se ha resuelto con Firebase, que aporta autenticación lista para usar, una base de datos NoSQL en tiempo real (Firestore) y reglas de seguridad declarativas, todo dentro del plan gratuito.

Esta elección elimina la necesidad de mantener servidores propios y permite enfocar el esfuerzo en la lógica de la aplicación.

1.3. Alcance del documento

Esta memoria documenta el proyecto técnico desde la puesta en marcha del entorno hasta el detalle de los procesos clave. Se acompaña del código fuente completo y de un README.md con instrucciones rápidas. Sigue la estructura sugerida por la Guía del Proyecto Fin de Ciclo facilitada por el centro.

2. Requisitos para el Desarrollo

En esta sección se detalla todo lo necesario para poner en marcha el proyecto en modo desarrollador, así como las dependencias externas que la aplicación utiliza.

2.1. Requisitos principales

Hardware mínimo recomendado

- Equipo de desarrollo: procesador de 64 bits, mínimo 8 GB de RAM (16 GB recomendados), 10 GB de espacio libre en disco.
- Dispositivo Android para pruebas (físico o emulador) con API 26 (Android 8.0 Oreo) o superior.
- Conexión a Internet estable para sincronizar con Firebase y cargar Google Maps.

Sistema operativo

- Compatible con Windows 10/11, macOS 10.14+ o Linux (Ubuntu 20.04+ recomendado).

Software necesario

Software	Versión	Uso
Android Studio	Iguana (2023.2.1) o superior	IDE principal
JDK	17 (incluido en Android Studio)	Compilación
Android Gradle Plugin	8.2.2	Build system
Kotlin	1.9.22	Lenguaje de la app
Gradle	8.5	Build system

Servicios en la nube

- Cuenta de Google con Firebase Console activado (gratuito, plan Spark).
- API Key de Google Maps generada desde Google Cloud Console (gratuita dentro de la cuota mensual).

2.2. Requisitos opcionales

La aplicación es plenamente funcional con los requisitos principales. Las siguientes opciones añaden valor pero no son imprescindibles:

- Dispositivo físico Android para probar la geolocalización real en lugar del emulador.
- Cuenta de pago en Google Cloud para superar las cuotas gratuitas si se hicieran muchas peticiones a Google Maps en producción.
- Servicio Stripe Connect para sustituir la pasarela simulada por pagos reales.

2.3. Dependencias

Las dependencias del proyecto se gestionan con Gradle (Kotlin DSL). Las principales son:

Dependencia	Función
Jetpack Compose BOM 2024.02.00	UI declarativa moderna sin XML
Material 3	Componentes visuales con paleta corporativa
Firebase BOM 32.7.2	Auth, Firestore (sin Storage para mantenerlo gratuito)
Maps Compose 4.3.0	Integración de Google Maps en pantallas Compose
Navigation Compose 2.7.7	Navegación entre pantallas con NavController
Coil 2.5.0	Carga de imágenes (URLs y Base64)
DataStore Preferences 1.0.0	Persistencia local del idioma y tema
Coroutines 1.7.3	Programación asíncrona (suspend functions)
Accompanist Permissions 0.32.0	Solicitud declarativa de permisos en Compose
Kotlinx Serialization JSON 1.6.0	Parseo del servicio de geocoding (Nominatim)

Servicios externos

- Firebase Authentication: gestiona usuarios y sesiones (email + contraseña).
- Cloud Firestore: base de datos en tiempo real para anuncios, reservas, avisos y transacciones de puntos.
- Google Maps SDK for Android: muestra mapas, marcadores y permite seleccionar ubicación tocando.
- Nominatim (OpenStreetMap): servicio de geocoding gratuito utilizado para autocompletar direcciones; se eligió frente a Google Geocoding por ser totalmente libre y sin API Key.

Orden de inicialización

La aplicación necesita que ciertos servicios estén disponibles antes que otros para arrancar correctamente:

1. Firebase debe estar inicializado antes de cualquier acceso a Firestore. Esto lo hace automáticamente el plugin google-services al leer google-services.json.
2. La API Key de Google Maps debe estar definida en gradle.properties antes de la compilación; si falta, los mapas se mostrarán en blanco.
3. DataStore se inicializa de forma diferida la primera vez que se accede al idioma guardado, en ParkableApp.attachBaseContext().

3. Puesta en marcha de la aplicación y uso

3.1. Estructura del código y archivos

El proyecto sigue una arquitectura por capas inspirada en el patrón MVVM (Model–View–ViewModel) con repositorios. Los paquetes están organizados por responsabilidad técnica para facilitar la localización del código:

```

app/src/main/java/com/parkable/app/
├── MainActivity.kt           ← única Activity; aplica locale en caliente
├── ParkableApp.kt           ← Application: lee idioma antes de cargar recursos
├── data/
│   ├── model/Models.kt     ← User, Listing, Booking, ParkingAlert, ...
│   ├── firebase/           ← FirebaseModule + Collections
│   ├── repository/         ← Auth, User, Listing, Alert (capa de datos)
│   └── GeocodingService.kt  ← cliente HTTP a Nominatim
├── locale/
│   ├── PreferencesRepository.kt ← DataStore (idioma, tema, onboarded)
│   └── LocaleHelper.kt      ← createConfigurationContext con el Locale
├── ui/
│   ├── theme/              ← Color, Type, Theme + gradientes
│   ├── components/         ← botones, cards, inputs reutilizables
│   ├── navigation/         ← Routes + ParkableNavHost (NavHost+bottombar)
│   └── screens/
│       ├── settings/       ← LanguageSelectionScreen, SettingsScreen
│       ├── auth/LoginScreen.kt
│       ├── home/HomeScreen.kt
│       ├── marketplace/    ← lista, detalle, publicar, pago (sector 1)
│       ├── socialdrive/    ← feed, nuevo aviso, detalle (sector 2)
│       ├── points/PointsScreen.kt
│       └── profile/ProfileScreen.kt
├── util/
│   ├── RewardsCatalog.kt   ← catálogo estático de recompensas
│   └── NotificationHelper.kt ← notificaciones locales
└── viewmodel/              ← AuthVM, ListingVM, AlertVM, PointsVM,
                             SettingsVM, AppViewModelFactory
  
```

Capas y responsabilidades

- **Capa de modelo (data/model):** data classes inmutables que mapean directamente con los documentos de Firestore. No contienen lógica.
- **Capa de repositorio (data/repository):** encapsula todo el acceso a Firebase. Expone Flows para que la UI reaccione a cambios en tiempo real, y funciones suspend para escrituras.
- **Capa de ViewModel (viewmodel):** adapta los datos al estado de UI, coordina llamadas a múltiples repositorios y aplica reglas de negocio (ej. reparto de puntos al completar un aviso).
- **Capa de UI (ui/screens):** pantallas Compose que solo leen de los ViewModels y emiten eventos. Sin acceso directo a Firebase.
-

3.2. Conceptos clave

A continuación se resumen las tecnologías más relevantes y cómo encajan entre sí:

Kotlin + Jetpack Compose

Compose es el framework de UI declarativa moderno de Android. La pantalla se describe como una función `@Composable` que toma estado y emite UI; cuando el estado cambia (flujos de Kotlin Coroutines), Compose recompone solo lo necesario. Esto elimina la necesidad de XML de layouts y de `findViewById`, simplifica los listeners y permite tests visuales más limpios.

Firebase como backend serverless

Firebase ofrece autenticación, base de datos NoSQL en tiempo real (Firestore) y reglas de seguridad sin necesidad de mantener un servidor. La aplicación usa exclusivamente el plan gratuito Spark, suficiente para un TFG y para una primera fase de validación con usuarios reales.

Firestore organiza la información en colecciones de documentos. Cada documento es un objeto JSON con tipos primitivos. Los repositorios subscriben listeners (`callbackFlow`) que emiten eventos cuando los datos cambian: la UI se actualiza sola sin necesidad de pull manual.

Google Maps Compose

Versión Compose-first del SDK de Maps. Permite incrustar un mapa como cualquier otro Composable, con cámaras, marcadores y detección de toques (`onMapClick`). Los anuncios y avisos se renderizan como marcadores; la creación interactúa tocando el mapa para fijar la ubicación exacta.

DataStore Preferences

Reemplazo moderno de `SharedPreferences`. Persiste el idioma elegido, el tema y la bandera de `onboarded` de forma asíncrona y reactiva (Flows). Se inicializa al primer acceso y no requiere bloquear el hilo principal.

Sistema de locales personalizado

La aplicación admite cambio de idioma en caliente sin reiniciar el sistema. Para conseguirlo, `ParkableApp.attachBaseContext()` lee el idioma guardado en `DataStore` y lo aplica a un contexto envuelto con `createConfigurationContext`. `MainActivity` observa cambios posteriores y llama a `recreate()`, de modo que toda la jerarquía Compose se reconstruye con los recursos del nuevo locale.

Imágenes en Base64 dentro de Firestore

Para mantener la app totalmente gratuita se evita `Firebase Storage` (que requiere plan Blaze). Las fotos de los anuncios se redimensionan a 400 px, se comprimen al 60% en JPEG y se guardan en Firestore como cadenas Base64 (`data:image/jpeg;base64,...`). Coil se encarga de decodificarlas para mostrarlas. Es un compromiso consciente: reduce drásticamente el tamaño individual a costa de no almacenar las imágenes a resolución original.

3.3. Base de Datos (Esquema)

La base de datos es Firestore, NoSQL orientada a documentos. Los datos se distribuyen en cinco colecciones que se relacionan por referencia (UID del usuario o id del documento padre).

Colección users

Campo	Tipo	Descripción
uid	String (PK)	Identificador único de Firebase Auth
name	String	Nombre completo introducido al registrarse
email	String	Email del usuario (único, validado por Firebase Auth)
photoUrl	String?	URL del avatar (opcional, no usado en MVP)
points	Int	Saldo de puntos canjeables
createdAt	Timestamp	Fecha y hora del alta

Colección listings

Campo	Tipo	Descripción
id	String (PK)	ID autogenerado por Firestore
ownerId	String (FK)	Referencia a users.uid del propietario
ownerName	String	Nombre denormalizado para evitar joins en lectura
title, description	String	Texto del anuncio
photos	List<String>	Mínimo 5 imágenes en formato data URI Base64
latitude, longitude	Double	Coordenadas de la plaza
address	String	Texto humano de la dirección
pricePerHour/Day/Week/Month	Double?	Tarifa por modalidad (cualquiera puede ser null)
available	Boolean	Visible en el listado público
createdAt	Timestamp	Fecha de publicación

Colección bookings

Campo	Tipo	Descripción
id	String (PK)	ID autogenerado
listingId	String (FK)	Anuncio reservado
renterId, renterName	String	Inquilino
ownerId	String	Propietario (denormalizado)
unit	Enum	HOUR / DAY / WEEK / MONTH
quantity, totalPrice	Int / Double	Cantidad de unidades y total cobrado
status	String	CONFIRMED por defecto en el MVP

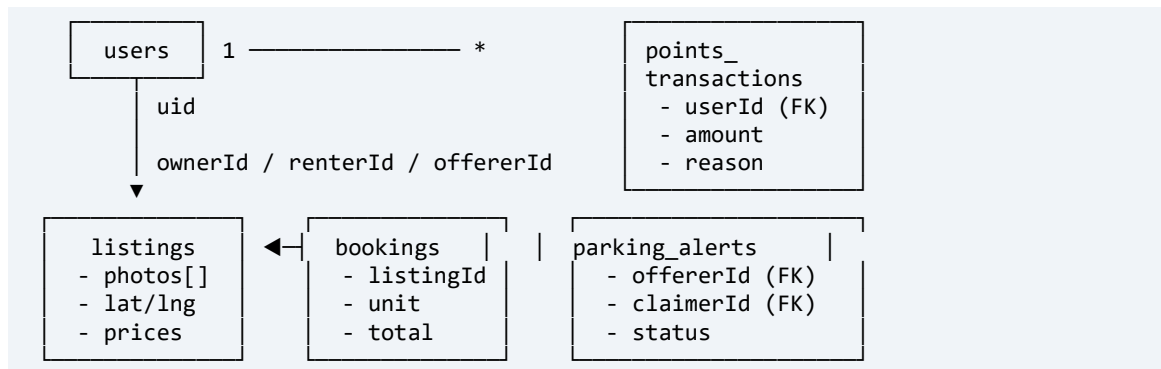
Colección parking_alerts

Campo	Tipo	Descripción
id	String (PK)	ID autogenerado
offererId, offererName	String	Conductor que sale
claimerId, claimerName	String?	Conductor que reclama (null hasta CLAIMED)
latitude, longitude	Double	Coordenadas de la plaza
addressHint, notes	String	Texto orientativo y notas
leavingInMinutes	Int	Margen anunciado por el ofreciente
status	Enum	OPEN, CLAIMED, COMPLETED, CANCELLED
offererConfirmed, claimerConfirmed	Boolean	Doble confirmación bilateral

Colección points_transactions

Cada movimiento (positivo o negativo) genera un documento en esta colección, que actúa como historial auditable. Los campos son: userId, amount (entero con signo), reason (cadena descriptiva como "alert_offered:abc123" o "redeem:car_wash") y createdAt.

Diagrama relacional simplificado



Modo de seguridad

Durante el desarrollo se usan reglas en "modo test" (acceso libre con sesión iniciada). Para producción se proporcionan en el código fuente reglas Firestore que limitan el acceso: cada usuario solo puede leer/modificar sus propios documentos en users y points_transactions; los listings son legibles por todos pero solo modificables por el ownerId; los parking_alerts solo permiten al offerer y al claimer escribir.

3.4. Procesos principales

Se documentan a continuación los flujos más representativos del funcionamiento de la aplicación. No se incluye todo el código, sino los puntos en los que la lógica es relevante para entender el sistema.

Flujo 1: registro y autenticación

- El usuario abre la app por primera vez. Al no haber sesión, ParkableNavHost.start = LANGUAGE.
- Tras seleccionar idioma, se muestra LoginScreen. El usuario alterna entre login y registro.
- AuthViewModel.register() llama a AuthRepository.signUp(): se crea el usuario en Firebase Auth y se inserta el documento User en Firestore con points = 0.
- AuthRepository.authStateFlow() emite el FirebaseUser → AuthViewModel.authedUser se actualiza → MainActivity navega a HOME.

Flujo 2: publicación de un anuncio

- El usuario llega a PublishListingScreen y selecciona al menos 5 fotos del galería (PickMultipleVisualMedia).
- Rellena título, descripción, dirección (con autocompletado vía AddressAutocompleteField → Nominatim) y toca el mapa para fijar las coordenadas.
- Establece precios para cada modalidad (cualquier combinación es válida).
- ListingViewModel.publishListing() valida los datos y llama a ListingRepository.uploadPhotos(): cada Uri se decodifica, redimensiona a 400 px, se comprime a JPEG 60% y se codifica en Base64.
- Las fotos en Base64 se incluyen en el documento Listing y se persisten con createListing().
- Como observeAvailableListings() está suscrito en MarketplaceViewModel, el nuevo anuncio aparece en la lista de todos los usuarios sin recargar.

Flujo 3: reserva con pago simulado

14. El usuario selecciona un anuncio en MarketplaceScreen y entra en ListingDetailScreen.
15. Elige modalidad y cantidad; el total se recalcula reactivamente.
16. Pulsa "Reservar y pagar" → navega a PaymentScreen pasando los argumentos por la ruta.
17. PaymentScreen muestra una tarjeta decorativa que se rellena en tiempo real mientras se escribe.
18. Al pulsar "Pagar X €" se ejecuta un delay(1500) para simular la latencia, después aparece la confirmación con CheckCircle.
19. ListingViewModel.book() crea el documento Booking en Firestore.
NotificationHelper.showBookingConfirmed() emite una notificación local con el título del anuncio.

Flujo 4: aviso SocialDrive con confirmación bilateral y entrega de puntos

20. Conductor A crea un aviso en NewAlertScreen: AlertViewModel.create() persiste un ParkingAlert con status = OPEN.
21. Todos los conductores ven el aviso en SocialDriveScreen gracias al snapshot listener de observeOpenAlerts().
22. Conductor B pulsa "Reclamar plaza": claimAlert() actualiza claimerId y cambia status a CLAIMED.
23. Cuando B llega y aparca pulsa "He aparcado" → confirmAsClaimer() pone claimerConfirmed = true.
24. Cuando A confirma la entrega → confirmAsOfferer() pone offererConfirmed = true; al detectar que ambas son true, status se actualiza a COMPLETED y la función devuelve true.
25. AlertViewModel.confirm() reparte los puntos: UserRepository.addPoints(offererId, +100) y addPoints(claimerId, +25), usando FieldValue.increment para que la operación sea atómica incluso con concurrencia.
26. Ambas operaciones generan un PointsTransaction en el historial, visible en PointsScreen.

Flujo 5: cambio de idioma en caliente

27. Usuario va a SettingsScreen y elige inglés.
28. SettingsViewModel.setLanguage("en") guarda en DataStore.
29. MainActivity observa languageFlow.distinctUntilChanged().drop(1) → recibe el nuevo valor → llama a recreate().
30. Al recrearse, attachBaseContext() vuelve a leer el idioma; LocaleHelper.wrap() devuelve un Context con Locale("en").
31. Compose se reconstruye y stringResource() devuelve los textos del archivo values-en/strings.xml.

Flujo 6: canje de recompensas

32. PointsScreen muestra el saldo del usuario y el catálogo definido en RewardsCatalog (4 recompensas: lavado 500, gasolina 750, premium 1500, café 200).
33. Al pulsar "Canjear" en una recompensa, PointsViewModel.redeem() comprueba el saldo.
34. Si es suficiente, llama a UserRepository.addPoints(uid, -coste, "redeem:rewardId").
35. Si no, devuelve un feedback que se muestra como Snackbar.
36. El historial se actualiza en tiempo real a través del Flow.

4. Posibles ampliaciones

La aplicación entregada constituye un MVP totalmente funcional. Existen varias direcciones de evolución que mejorarían el producto si se quisiera llevar a producción real:

4.1. Ampliaciones técnicas

- **Pasarela de pago real con Stripe Connect:** sustituir el flujo simulado por integración con Stripe, que gestiona la separación de fondos entre conductores y propietarios automáticamente.
- **Mensajería interna y notificaciones push:** añadir Firebase Cloud Messaging para que los conductores puedan chatear cuando una reserva es inminente o un aviso queda reclamado.
- **Filtros geográficos avanzados:** añadir consultas por proximidad usando GeoFirestore o GeoHashes, de forma que la app muestre solo plazas en un radio configurable alrededor del usuario.
- **Almacenamiento de imágenes en CDN:** al pasar a un plan Blaze de Firebase, mover las fotos de Base64 a Cloud Storage para servir las más eficientemente.
- **Tests automatizados:** añadir suites de tests unitarios para repositorios y ViewModels (JUnit + Coroutines Test) y tests de integración con Compose Test Rule.
- **Inyección de dependencias con Hilt:** sustituir AppViewModelFactory por Hilt para escalar cuando el grafo de dependencias crezca.

4.2. Ampliaciones funcionales

- **Sistema de reseñas y reputación:** permitir a los conductores valorar a los propietarios y viceversa tras cada reserva.
- **Reservas con calendario:** en lugar de seleccionar cantidad de unidades, permitir elegir fechas concretas con un date picker; bloquear el anuncio en esas fechas para evitar dobles reservas.
- **Versión web complementaria:** replicar la app como aplicación web (Compose for Web o React) para los propietarios B2B (administradores de fincas), donde una pantalla grande es más cómoda que un móvil.
- **Integración con vehículos:** en colaboración con fabricantes, abrir la barrera del garaje desde la app vía API del vehículo.
- **Dashboard B2B con analíticas:** construir una sección para administradores con métricas de ocupación, ingresos y exportación a Excel.

4.3. Ampliaciones de gamificación

- **Niveles y badges:** añadir progresión visual con insignias por hitos (10 avisos completados, 100 puntos canjeados, etc.).
- **Recompensas con marcas reales:** establecer acuerdos con cadenas de gasolineras o cafeterías para que los puntos canjeables sean cupones reales generados al confirmar el canje.

5. Valoración personal

El desarrollo de Parkable ha supuesto un reto integral en el que se han aplicado prácticamente todos los contenidos del ciclo: programación orientada a objetos con Kotlin, programación asíncrona con coroutines y flows, persistencia local y en la nube, integración con APIs externas, internacionalización, diseño de interfaces y arquitectura de software. Más que producir una app funcional, el objetivo ha sido construirla con criterios de calidad cercanos a un producto real.

5.1. Mayores dificultades encontradas

Cambio de idioma en caliente

Cambiar el idioma de una app Android sin reiniciarla es un caso clásico de fricción. La solución habitual (override en `attachBaseContext`) requiere coordinar `Application`, `Activity` y `Compose`. Conseguir que un cambio en `DataStore` propagara hasta los `stringResource()` de cada pantalla sin parpadeos llevó varias iteraciones, y se resolvió combinando `attachBaseContext` con un observador en `MainActivity` que llama a `recreate()` cuando detecta cambios. Esta arquitectura es la que permite que tanto el selector de idioma de inicio como el botón de Ajustes funcionen idénticamente.

Mantener todo en el plan gratuito

Firebase Storage está fuera del plan Spark a partir de cierta cantidad de operaciones, lo que obligó a buscar una alternativa para almacenar las fotos de los anuncios. La solución (codificar a Base64 y guardar en Firestore) tiene el coste de un mayor uso de bytes por documento, pero asegura que el proyecto sea reproducible al 100% sin tarjeta de crédito ni alta de servicios de pago. Implementarlo requirió escribir el redimensionado y compresión a mano (`BitmapFactory` + `Bitmap.compress`), un buen ejercicio de manejo de imágenes en Android.

Doble confirmación bilateral

El flujo de avisos de SocialDrive exige que dos usuarios distintos confirmen la misma transacción para que se completen los puntos, algo que en una BBDD sin transacciones cliente-servidor podría dar lugar a inconsistencias. Se resolvió leyendo el documento tras cada `update` y verificando atómicamente si las dos banderas booleanas eran `true`. El reparto de puntos usa `FieldValue.increment`, que es atómico en Firestore incluso con clientes concurrentes.

Geocoding gratuito

Google Geocoding API es de pago. Para el autocompletado de direcciones se utilizó Nominatim (OpenStreetMap), que es libre, pero con limitaciones de velocidad. Hubo que implementar un `debounce` (delay de 500 ms) para no saturar el servicio cuando el usuario escribe rápido.

5.2. Aprendizajes clave

- **Compose cambia la forma de pensar la UI:** pasar de un mundo imperativo (XML + findViewById) a uno declarativo (estado → UI) ha sido el aprendizaje técnico más importante. Aplicar inmutabilidad y unidirectional data flow simplifica la lectura del código.
- **La separación en capas paga rápido:** haber aislado los repositorios desde el principio permitió cambiar Storage por Base64 sin modificar la UI.
- **El producto se diseña antes de codificar:** el análisis sectorial previo (panorama TIC andaluz, priorización por urgencia/alcance/impacto, DAFO, MoSCoW) fue clave para no escribir funciones que después había que descartar. La trazabilidad RF→código es directa.
- **Los detalles cuentan:** los pequeños retoques de UX (gradientes corporativos, animación del pago, confirmación visual del canje, badge de puntos en perfil) marcan la diferencia entre una app de prácticas y un MVP creíble.

5.3. Conclusión

Parkable cumple los objetivos planteados: es funcional, está respaldada por una arquitectura limpia, soporta multilingüismo, integra servicios reales (Firebase, Maps, geocoding) y mantiene un coste cero. Su evolución natural hacia un producto comercial pasaría por las ampliaciones señaladas (pago real, mensajería, GeoFirestore), pero como entrega de fin de ciclo refleja con honestidad el nivel técnico que el grado pide. He disfrutado especialmente la parte de UI con Compose y el diseño del sistema de puntos, donde se mezclan datos en tiempo real, atomicidad y feedback visual.

6. Apéndices

6.1. Material adicional incluido

Junto con esta memoria se entregan los siguientes recursos:

- Código fuente completo del proyecto en formato ZIP, con estructura estándar de Android Studio.
- README.md con instrucciones de puesta en marcha rápida y descripción técnica resumida.
- Documentación previa del análisis sectorial andaluz (Panorama Sectorial TIC) y de la oportunidad seleccionada (Movilidad Urbana Inteligente).
- Documento C1_5_6 con la definición del tipo y características del proyecto, que sirvió de base para el desarrollo.

6.2. Pasos para reproducir el proyecto

37. Crear un proyecto en Firebase Console con package name com.parkable.app y descargar el archivo google-services.json.
38. Activar Firebase Authentication (Email/Password) y Cloud Firestore en modo test.
39. Generar una API Key de Google Maps en Google Cloud Console, habilitando "Maps SDK for Android".
40. Pegar la API Key en el archivo gradle.properties (variable MAPS_API_KEY).
41. Reemplazar el archivo google-services.json del directorio app/ por el descargado.
42. Sincronizar Gradle en Android Studio y ejecutar la app.

6.3. Tecnologías y sus versiones (resumen)

Componente	Versión
Android compileSdk / targetSdk	34
Android minSdk	26 (Android 8.0)
Kotlin	1.9.22
Android Gradle Plugin	8.2.2
Compose Compiler	1.5.8
Firebase BOM	32.7.2
Maps Compose	4.3.0

6.4. Referencias y enlaces de interés

- Documentación oficial de Jetpack Compose: <https://developer.android.com/jetpack/compose>
- Documentación de Firebase Android: <https://firebase.google.com/docs/android/setup>
- Maps Compose: <https://github.com/googlemaps/android-maps-compose>
- Nominatim (OpenStreetMap): <https://nominatim.org/release-docs/latest/api/Overview/>
- Material Design 3: <https://m3.material.io/>
- Coil (image loading): <https://coil-kt.github.io/coil/>

6.5. Antonio

Antonio Raño - Alumno de 2.º curso del Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Multiplataforma.

Centro: IES El Majuelo.

Curso académico: 2025-2026.